Fair Influence Maximization with Reverse Influence Sampling Boosted Multi-Objective Genetic Algorithms

A Project

Presented to

The Faculty of the Department of Computer Science

San José State University

In Partial Fulfillment

of the Requirements for the Degree

Master of Science

by

Akash Janardhan Srinivas

December 2025

© 2025

Akash Janardhan Srinivas

ALL RIGHTS RESERVED

The Designated Project Committee Approves the Project Titled

Fair Influence Maximization with Reverse Influence Sampling Boosted Multi-Objective Genetic Algorithms

by

Akash Janardhan Srinivas

APPROVED FOR THE DEPARTMENT OF COMPUTER SCIENCE

SAN JOSÉ STATE UNIVERSITY

December 2025

Dr. Katerina Potika	Department of Computer Science
Dr. William Andreopoulos	Department of Computer Science
Rohan Mohapatra	Founding engineer, Llmda AI

ABSTRACT

Fair Influence Maximization with Reverse Influence Sampling Boosted Multi-Objective Genetic Algorithms

by Akash Janardhan Srinivas

Influence Maximization (IM) focuses on identifying a small set of influential nodes (seed nodes) in social networks to maximize the spread of information, ideas, or behaviors efficiently. Traditional IM methods prioritize overall influence spread, neglecting fairness considerations regarding who benefits from that spread. This oversight might result in particular disadvantaged demographic groups being omitted from seed selection and thus receiving critical information later, or not at all. This raises ethical concerns in scenarios involving health, education, or employment resources.

Fair Influence Maximization (FIM) seeks to ensure equitable diffusion across predefined groups. While early FIM approaches, such as max–min fairness, address disparities in a simple way, they fail to capture subtler inequities. Recent frameworks incorporate statistical measures such as the Gini coefficient to evaluate inequality across influenced nodes.

This work introduces FIMMOGA++, an enhanced genetic algorithm that integrates multi-objective optimization and fairness-aware design. It combines traditional IM heuristics with Reverse Influence Sampling (RIS) and the Non-dominated Sorting Genetic Algorithm III (NSGA-III) to simultaneously optimize influence spread and multiple fairness criteria. By producing a Pareto-optimal set of solutions, FIMMOGA++ provides balanced strategies that advance both theoretical understanding and practical implementation of fair influence maximization in large-scale networks.

Keywords: Influence Maximization, Fairness, Social Network Analysis, Gini Coefficient, Multi-objective Optimization, Genetic Algorithms, Pareto Front, Information Dissemination

ACKNOWLEDGMENTS

I am deeply grateful to my project advisor, Dr. Katerina Potika, for her guidance, patience, and constant support throughout this work. She helped me shape the research problem, refine the methodology, and improve the writing at every stage of the project. Her careful feedback and steady encouragement made this thesis possible.

I would also like to thank Prof. William Andreopoulos and Rohan Mohapatra for their helpful comments and suggestions on my work.

TABLE OF CONTENTS

CHAPTER

1 Introduction 1

1.1 Background and Terminologies 2

2 Related Work 7

2.1 Classical Influence Maximization 7

2.2 Early Fair Influence Maximization 9

2.3 Fairness Metrics and Distributional Perspectives 9

2.4 Multi-Objective and Evolutionary Approaches 10

2.5 Positioning of the Present Work 11

3 Methodology 12

3.1 Datasets and Group Construction 12

3.2 Baseline Methods 13

3.3 Fairness Objectives 14

3.3.1 Inequality (One Minus Gini) 14

3.3.2 Max--Min Coverage 14

3.3.3 Parity Relative to Equal Treatment 14

3.3.4 Isoelastic Welfare 15

3.3.5 Spread Objective and Normalization 15

3.4 Reverse Influence Sampling and Fitness 16

3.4.1 RR Set Construction 17

3.4.2 RIS Based Fitness 17

3.5 Genetic Algorithm and Many Objective Selection 18
3.5.1 Initialization with Prior Knowledge 18
3.5.2 Fitness evaluation via RIS 19
3.6 NSGA based Selection and GA Cycle 19
3.6.1 Overview of the GA cycle 19
3.6.2 Non dominated sorting and reference points 20
3.7 Variation Operators for Seed Sets 21
3.7.1 Uniform set crossover 21
3.7.2 Community aware mutation 22
3.7.3 Group budget repair 22
3.7.4 Local one swap improvement 23
3.8 Overall FIMMOGA++ Loop 23
3.9 Complexity and Scalability 23

4 Methodological Details 27
4.1 Group Construction and Robustness 27
4.2 Objective Vector and Normalization 27
4.3 Genetic Algorithm for Seed Set Search 28
4.3.1 Chromosome representation of seed sets 28
4.3.2 Initial population with prior knowledge 29
4.3.3 GA configuration 29
4.4 RIS-Based Fitness and Computational Cost 30

5 Experimental Design and Results 31
5.1 Experimental Design 31

5.1.1 Datasets 31
5.1.2 Methods Compared 32
5.1.3 Experimental Protocol 32
5.1.4 Metrics 33
5.1.5 Sensitivity and Ablations 34
5.2 Results 36
5.2.1 Overall Comparison Across Datasets 36
5.2.2 Runtime Comparison 37
5.2.3 Sensitivity to Seed Budget k 39
5.2.4 Sensitivity to Propagation Probability p 40
5.2.5 Ablation Studies 41
5.2.6 Summary of Findings 44
6 Conclusion and Future Work 46
6.1 Summary of Contributions 46
6.2 Main Findings 46
6.3 Limitations 47
6.4 Future Work 48
LIST OF REFERENCES 49
APPENDIX

LIST OF TABLES

1	Datasets used in the study and their basic statistics.	31
2	Ablation grid: what is varied, what is held fixed, and what we measure.	35
3	Influence and fairness across methods (final Monte Carlo evaluation; k=20, p=0.10).	36
4	Runtime comparison (seconds) at k=20, p=0.10 on three datasets; total time includes seed selection and final Monte Carlo evaluation.	39
5	FIMMOGA++ across seed budgets (p=0.10, RIS Θ=100,000). .	40
6	FIMMOGA++ versus CELF across propagation probabilities $p \in \{0.05, 0.10, 0.20\}$ (k=20).	42
7	MC versus RIS fitness inside the genetic algorithm (k=20, p=0.10).	42
8	Selector comparison by number of objectives: NSGA-II with two objectives versus NSGA-III with five.	43
9	Effect of RIS budget Θ on FIMMOGA++ (k=20, p=0.10). . . .	44

LIST OF FIGURES

1 Roadmap: high-level view of our approach to fair influence maximization. 5

2 Conceptual map of the influence maximization and fair IM literature. 8

3 Fairness objectives used in this work, illustrated via group-level coverage patterns. 16

4 Step-by-step overview of one GA iteration from initial population through parent selection, variation, and replacement. 20

5 High-level flow of FIMMOGA++ from input and RIS preprocessing to GA search and final evaluation. 25

6 High-level flow of techniques in this work. 26

7 Spread versus Max--Min coverage for each dataset; each point is one method, with FIMMOGA++ shown as a star. 38

8 Total runtime (log scale) for CELF, original FIMMOGA, and FIMMOGA++ on three datasets (k=20, p=0.10). 39

9 Sensitivity of FIMMOGA++ to the seed budget k (p=0.10, RIS budget Θ=100,000). 41

10 Sensitivity of spread to propagation probability p (k=20) for CELF (solid squares) and FIMMOGA++ (dashed stars). 43

CHAPTER 1

Introduction

The rapid growth and complexity of social networks have made influence maximization (IM) a fundamental problem in computational social science, marketing, public health, and policy making. Platforms such as Facebook, Twitter, and LinkedIn enable information to spread quickly through large populations. The IM problem was first formalized by Kempe et al. [1]. It asks for a set of k seed nodes that maximizes expected influence under a diffusion model such as the Independent Cascade (IC) or Linear Threshold (LT) model.

Classical IM methods are highly effective at maximizing overall reach but largely ignore who benefits from that reach. When the only goal is to maximize total spread, a strategy may concentrate influence in dense cores and underserve minority or peripheral communities. In practical settings, this may mean delayed access to healthcare information, job opportunities, or public services for groups that are already disadvantaged. Recent work on Fair Influence Maximization (FIM) responds to this concern by introducing explicit fairness criteria. It aims that the benefits of diffusion are shared more equitably across predefined groups such as demographic categories, departments, or communities detected from the network [2, 3, 4, 5].

Ensuring fairness in IM is not straightforward. On the modeling side, we need metrics that capture how influence is distributed across multiple groups, rather than a single global count. On the algorithmic side, we must deal with several objectives that can conflict with each other, for example, maximizing spread while also improving coverage of the worst served groups. Fairness formulations such as maximin coverage, equality of influence, or social welfare have been proposed in the literature, and each captures a different view of what it means for an allocation of influence to be fair [2, 4]. This motivates treating fairness as a vector of objectives instead of a single constraint.

In this work, we build on multiobjective genetic algorithms (GAs) as a flexible way to navigate these trade-offs. We adopt an enhanced approach that combines four ideas. First, we replace the slow Monte Carlo (MC) simulation in the inner evaluation loop with **Reverse Influence Sampling (RIS)**, which estimates spread using coverage over a set of pre-sampled reverse reachable (RR) sets [6, 7]. This gives a fast and low-variance fitness signal for the GA. Second, we treat **multiple fairness notions** as first-class objectives, including inequality, worst group coverage, parity, and welfare inspired by isoelastic social welfare models [4, 8]. Third, we use **Non-dominated Sorting Genetic Algorithm III (NSGA-III)** [9] to maintain diversity across these many objectives, extending the ideas of NSGA-II [10] to higher dimensions. Fourth, we incorporate **domain-informed operators** so that the search respects group structure and remains computationally easy on graphs. This is tested on Cora, Email-Eu-core, Facebook, CollegeMsg, and OpenFlights North America datasets. This combination yields consistent gains in fairness with modest and controllable losses in spread. In some cases, even an improved spread compared to baselines was observed. The overall design is also informed by earlier work on FIMMOGA [5], which combines genetic search and fairness in influence maximization.

1.1 Background and Terminologies

Influence Maximization (IM). Given a graph $G = (V, E)$, a seed budget k, and a diffusion model, IM asks us to select a seed set $S \subseteq V$ with $|S| = k$ that maximizes the expected number of activated nodes,

$$\max_{S \subseteq V,\, |S|=k} \mathbb{E}[\sigma(S)],$$

where $\sigma(S)$ denotes the random spread under the chosen diffusion model [1].

Fair Influence Maximization (FIM). Fair IM extends the classical problem by introducing equity across predefined groups. The node set V is partitioned into groups

$G_1, \ldots, G_m$ (for example, demographic categories, departments in an organization, or communities obtained by a clustering algorithm). Instead of a single objective, we now care about both high total spread and a fair distribution of influence among these groups [2, 3, 4, 5].

Seed Nodes. Seed nodes are the nodes initially activated at time zero. They act as starting points for the diffusion process. The choice of seeds is the main degree of freedom in IM and FIM.

Fairness metrics. Let $f_g(S) \in [0, 1]$ denote the expected fraction of group g that is influenced by seed set S. We use several complementary notions of fairness, inspired by recent work on fair influence maximization and social welfare [2, 4]:

- Inequality (Gini): We compute the Gini coefficient of the vector $(f_1(S), \ldots, f_m(S))$ and report $(1 - \text{Gini})$ so that higher values indicate more equal coverage.
- Max–Min coverage: The quantity $\min_g f_g(S)$ captures how well the worst served group is doing, in the spirit of maximin fairness.
- Parity: We measure deviations from equal treatment across groups via $(1 - \max_g |f_g(S) - \bar{f}(S)|)$, where $\bar{f}(S)$ is the average group coverage. Larger values indicate smaller gaps between groups.
- Welfare: We compute an isoelastic social welfare measure $\frac{1}{m} \sum_{g=1}^{m} f_g(S)^\alpha$ with $0 < \alpha \leq 1$, which assigns more weight to improvements in underserved groups.

These metrics highlight different aspects of fairness and, taken together, give a richer picture than any single index.

Multiobjective optimization. When we include fairness, IM naturally becomes a multiobjective optimization problem. Instead of searching for a single best seed set, we search for a set of Pareto optimal seed sets that are not dominated on any

of the objectives. A solution A dominates a solution B if A is at least as good as B on all objectives and strictly better on at least one objective. The collection of all non-dominated solutions forms a Pareto frontier, which makes the trade-offs between spread and different fairness notions explicit [10].

Prior knowledge. Simple heuristics such as Degree, Closeness, Betweenness, and PageRank centrality tend to identify nodes that are structurally important in the graph and often make strong seed candidates. We use these heuristic seed sets as prior knowledge by injecting them into the initial GA population, along with random and per-group heuristic seeds, to accelerate convergence and reduce the risk of the GA spending many generations on poor quality regions of the search space.

Reverse Influence Sampling (RIS). RIS replaces repeated forward diffusion simulations with coverage over reverse reachable (RR) sets. Each RR set is generated by selecting a node at random, then simulating the diffusion process in reverse on a randomly sampled live edge graph. The expected spread of a seed set S can be estimated by the fraction of RR sets that intersect S [6, 7]. By tagging each RR set with the group of its root node, we can also estimate the group level coverage $f_g(S)$ without additional simulation cost. This provides a fast and low-variance fitness signal for the GA.

NSGA-III. NSGA-III is a many-objective evolutionary algorithm that extends NSGA-II to settings with four or more objectives [10, 9]. Instead of relying solely on crowding distance, NSGA-III uses a set of predefined reference directions in the objective space and assigns solutions to these directions to maintain diversity. In our setting, NSGA-III helps keep a well-spread set of solutions in the five-dimensional space defined by spread, inequality, max–min coverage, parity, and welfare.

Figure 1 summarizes the high-level flow of the thesis: it starts from the motivation

for fairness in IM, moves through related work and the design of FIMMOGA++, and ends with experiments, results, and conclusions.

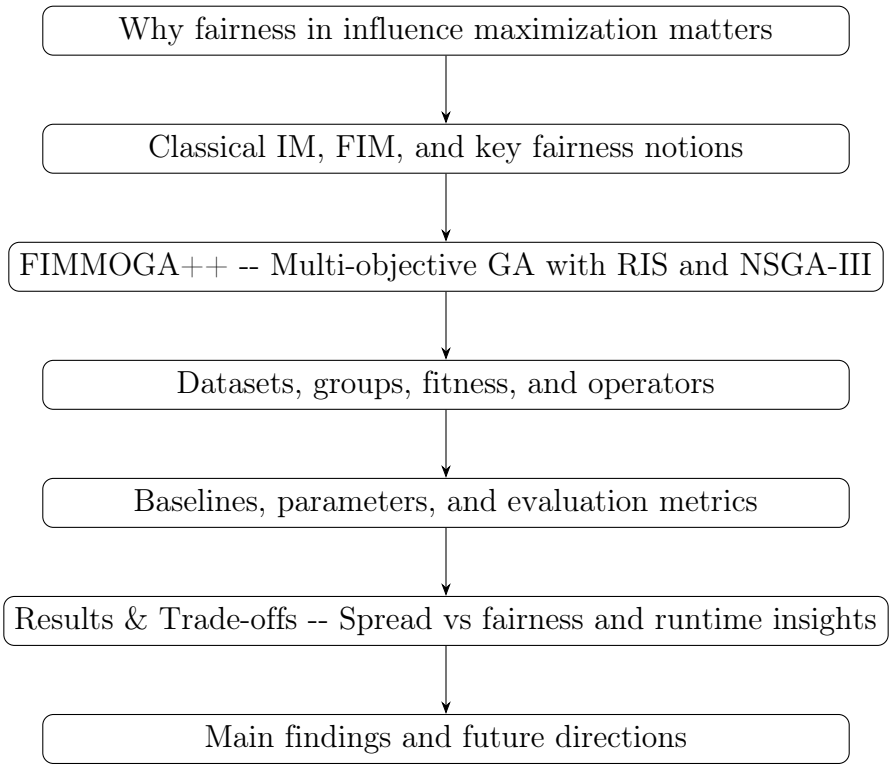


Figure 1: Roadmap: high-level view of our approach to fair influence maximization.

The remainder of this thesis is organized as follows. Chapter 2 reviews related work on classical influence maximization, fairness aware extensions, and evolutionary approaches, with particular attention to FIMMOGA and other fair IM frameworks that motivate our design choices [1, 2, 3, 4, 5]. Chapter 3 and Chapter 4 describe the methodological details of our approach, including the construction of group labels, the definition of fairness metrics, the RIS based fitness estimator, and the NSGA-III based genetic algorithm with domain informed operators [6, 7, 10, 9, 5]. Chapter 5 presents the experimental design and implementation, covering datasets, baseline methods, parameter settings, and the evaluation protocol. It also reports the results of our experiments, including overall performance, sensitivity analyses, and ablation studies

that isolate the effect of RIS, many objective optimizations, and operator choices. Finally, Chapter 6 summarizes the main findings, discusses limitations of the current approach, and outlines directions for future work on scalable and robust fair influence maximization.

CHAPTER 2

Related Work

The evolution of influence maximization (IM) and fair influence maximization (FIM) is an important development in network analysis and algorithmic decision making. Classical IM work focuses on maximizing the size of the cascade under diffusion models such as Independent Cascade (IC) and Linear Threshold (LT). Threshold-based diffusion was studied earlier in sociology [11], and later work gave these models strong approximation guarantees and highly optimized implementations [1, 12, 7]. Influence as viral marketing over customer networks was also explored in early data mining work [13]. However, these methods rarely consider who receives the influence, and can unintentionally favor structurally central or majority communities.

Fair influence maximization introduces equity goals on top of spread [2, 3, 4, 5]. Rather than treating fairness as an afterthought, this work regards equity as a core objective. It asks how seed sets should be chosen so that the diffusion is shared more evenly across groups or communities. Our work builds on this direction and also advances in reverse sampling, multi-objective optimization, and evolutionary algorithms. Figure 2 provides a visual summary of how the literature evolves from classical IM to scalable sampling, early fair IM, and multi-objective GA-based approaches like FIMMOGA and FIMMOGA++.

2.1 Classical Influence Maximization

The modern formulation of IM is due to Kempe et al. [1]. They studied the IC and LT models and showed that the expected spread function is monotone and submodular. This result justified a simple greedy algorithm with a $(1 - 1/e)$ approximation guarantee. It iteratively added the node with the largest marginal increase in expected spread. Later work focused on scaling this idea to large graphs. Common techniques include lazy evaluation (CELF), degree-based heuristics, and refined

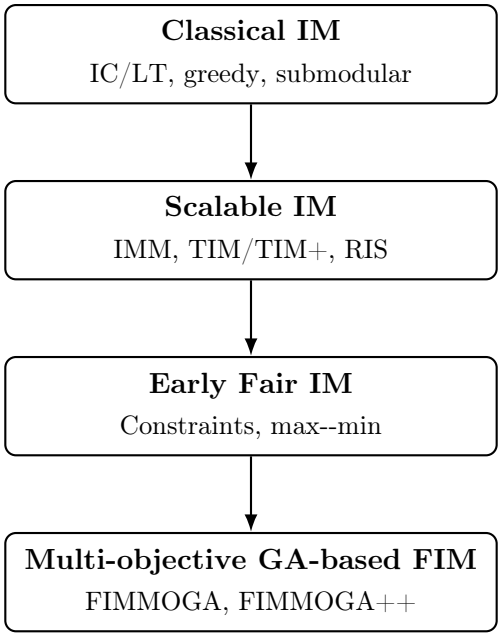


Figure 2: Conceptual map of the influence maximization and fair IM literature.

diffusion approximations [14, 12, 7].

Scalable sampling methods such as IMM and TIM/TIM+ combine theoretical guarantees with practical speed [12, 7]. They use martingale concentration and carefully chosen samples of diffusion traces. Reverse influence sampling (RIS) is central to these methods. Instead of simulating forward cascades many times, one draws reverse reachable sets and chooses seeds that cover as many of these sets as possible. These advances make it feasible to run IM on graphs with millions of nodes. However, they still treat all nodes as interchangeable from a fairness perspective.

A common theme in this classical work is objective myopia. The only quantity being optimized is total spread, and the output is a single seed set that maximizes it. The distribution of benefits across groups, and the possibility of structural bias in the resulting cascades, are not examined.

2.2 Early Fair Influence Maximization

Early work on fair IM proposed ways to inject equity into the objective or constraints. One family of approaches uses max--min criteria [2, 3]. The goal is to maximize the minimum influenced fraction among all groups, or to guarantee that the least advantaged group reaches a given coverage. This prevents extreme neglect. However, it does not distinguish between solutions that tie on the worst case yet may differ a lot for the rest of the population.

Another family introduces explicit fairness constraints [2]. For example, one can require that each protected group achieves at least a threshold level of influence, or enforce budget splits so that each group receives at least one or a fixed number of seeds. These designs are simple and easy to explain. They are natural when there are legal or policy thresholds. However, they are relatively coarse. Once the threshold is met, the optimization no longer distinguishes between more or less equitable distributions. Overly strict constraints can also damage total spread.

These contributions opened up the space of fair IM and showed that pure spread maximization may not be acceptable in many settings. Groups can differ in size, position in the network, or historical access to information. In such cases, fairness is inherently a multi-objective concern rather than a single scalar constraint.

2.3 Fairness Metrics and Distributional Perspectives

Subsequent work began to consider fairness in a more distributional way. Rather than focusing only on the worst group or on hard thresholds, these approaches examine the entire vector of group level influenced fractions and seek metrics that reflect its shape [2, 4, 5]. The Gini coefficient, originally introduced as a measure of economic inequality [15], has been adapted to measure how evenly influence is distributed across groups [5]. A low Gini value indicates that groups receive comparable influence. A

high value signals large disparities.

Other notions have also been proposed. Examples include max--min coverage, parity gaps relative to equal treatment, and social welfare based on concave utilities. Each metric captures a different normative view of fairness. No single metric covers every concern, and optimizing for one can sometimes harm another. This motivates multi-objective formulations in which several fairness measures are optimized together with spread. More broadly, the study of fairness in algorithms and machine learning has developed a rich set of concepts and trade-offs [16, 17]. In this thesis, the detailed definitions of these metrics and their use as objectives are given in Chapter 3.

2.4 Multi-Objective and Evolutionary Approaches

The introduction of fairness turns IM into a multi-objective optimization problem. Instead of a single optimum, the goal is to identify a set of Pareto optimal seed sets. A seed set is Pareto optimal if it cannot be improved on one objective without worsening another. Well-known multi-objective evolutionary algorithms, such as NSGA-II [10] and NSGA-III [9], have been developed to handle such trade-offs. They use populations of candidate solutions, evolve them with variation operators, and select non-dominated individuals while maintaining diversity in objective space. General introductions to multi-objective evolutionary optimization can be found in [18], and classical genetic algorithm ideas are surveyed in [19].

Multi-objective evolutionary strategies are attractive for fair IM for several reasons. They can handle non-convex and discontinuous Pareto fronts. They do not require explicit gradients. They also support multiple fairness metrics at once in a natural way. FIMMOGA [5] is a prominent example. It combines prior knowledge in the form of centrality-based seeds with a genetic algorithm that optimizes total spread and a Gini-based fairness measure, using Monte Carlo simulation for fitness evaluation. There

is also work on evolutionary approaches to IM without explicit fairness constraints, which focuses on spread and structural features of the graph [20, 21].

2.5 Positioning of the Present Work

Our work is a direct continuation and refinement of the FIMMOGA line. We keep the idea of using a genetic algorithm with prior knowledge and a fairness-aware objective, but we modify several key components:

- we adopt reverse influence sampling (RIS) [7] to estimate spread and group-wise coverage, replacing expensive inner loop Monte Carlo;
- we treat multiple fairness notions (inequality, max--min, parity, welfare) as separate objectives rather than collapsing them into a single scalar;
- we move from NSGA-II to NSGA-III [10, 9] to better handle a five dimensional objective space; and
- we introduce simple domain-informed operators that respect group structure and community information.

Empirical evaluation in this thesis is carried out on five real networks: Cora, Email-Eu-core, SNAP Facebook, CollegeMsg, and an OpenFlights-based North America network [22, 23]. This combination covers citation, organizational email, online social interaction, message exchange, and transportation graphs. The formal definition of the objectives, the RIS-based fitness estimator, and the genetic search procedure are presented in detail in Chapter 3. We also describe the implementation decisions that enable efficient experiments on these datasets.

CHAPTER 3

Methodology

This chapter describes the design of our fair influence maximization framework, which we refer to as FIMMOGA++. The method combines reverse influence sampling (RIS) for fast and stable fitness estimates [6, 7], several fairness metrics inspired by prior work on fair IM [2, 3, 4, 5, 15], and a many objective genetic algorithm based on NSGA-III [10, 9]. We also explain how prior knowledge from standard heuristics is used to seed the search. Finally, we describe how the implementation remains compatible with the original FIMMOGA code base [5].

3.1 Datasets and Group Construction

We work with five real networks:

- **Cora (citation).** Papers are nodes and citation links are edges. Groups are defined by paper class [22].
- **Email-Eu-core.** Nodes are users in a European research institution. Groups are defined by department labels [23].
- **SNAP Facebook.** Ego networks are combined into a larger Facebook friendship graph. Groups are defined by community detection (greedy modularity as the main partition, with label propagation only for robustness checks) [23].
- **CollegeMsg (SNAP).** This is a temporal network of private messages between users in an online social network. We ignore timestamps and treat it as a simple undirected graph. Groups are defined by community detection on the aggregated graph [23].
- **OpenFlights NA.** This is a subgraph of the OpenFlights airline network restricted to North America, with airports as nodes and flight routes as edges. Groups are defined by country or region labels and, when needed, refined by community detection.

All networks are treated as simple undirected graphs for the IC model unless a directed variant is explicitly stated. Each node is assigned to a single group, using the provided label when available or the community assignment otherwise. This grouping policy is part of the experimental design. Its implications for fairness are revisited in Chapter 5.

3.2 Baseline Methods

We evaluate FIMMOGA++ against several strong and interpretable baselines:

- **Heuristic seed selection.** We take the top-k nodes by Degree, Closeness, Betweenness, or PageRank centrality, computed on the undirected graph [1, 14, 24, 25]. These are standard centrality measures in network analysis and web graphs.
- **Per group top degree.** This is a simple fairness aware heuristic. It first chooses the highest degree node in each group, then fills remaining seeds from the global degree ranking.
- **Greedy CELF.** This is a spread oriented method that approximates the classical greedy algorithm using lazy evaluations and Monte Carlo simulations under the IC model [26].
- **RIS greedy.** This is a reverse sampling based method that greedily selects seeds to maximize coverage of reverse reachable sets, following the ideas in IMM/TIM style algorithms [12, 7].
- **Original FIMMOGA.** This is the method of Gong and Guo [5]. It uses prior knowledge and a genetic algorithm with Monte Carlo based fitness and a single Gini based fairness objective.

These baselines form the comparison set across all five datasets. Spread only methods show what can be achieved when equity is ignored. Fairness aware heuristics

show the effect of simple rules. The original FIMMOGA provides a direct point of comparison for our modifications.

3.3 Fairness Objectives

Let $G = (V, E)$ be a graph with node set V, and let $\mathcal{G}$ denote the set of groups. For a seed set $S \subseteq V$ and a diffusion process under IC, we write $f_g(S)$ for the expected fraction of group g that becomes influenced by the end of the cascade. Our framework considers four fairness objectives. Each objective captures a different view of how the influence is spread across groups. The way in which $f_g(S)$ is estimated from RR sets is described in Section 3.4.

3.3.1 Inequality (One Minus Gini)

The Gini coefficient [15] is a standard measure of inequality. Given the vector of group level coverage values $\{f_g(S) : g \in \mathcal{G}\}$, the Gini coefficient $\mathrm{Gini}(S)$ lies in $[0, 1]$. A value of 0 indicates perfect equality. Values closer to 1 indicate more uneven coverage. To fit our maximization setting we use

$$J_{\mathrm{ineq}}(S) = 1 - \mathrm{Gini}(S),$$

so that higher values correspond to more equal coverage across groups.

3.3.2 Max--Min Coverage

To protect the worst served group, we consider the max--min coverage objective

$$J_{\min}(S) = \min_{g \in \mathcal{G}} f_g(S).$$

This measure is sensitive to extreme neglect and matches earlier max--min fairness formulations [2, 3]. On its own it does not distinguish between solutions that tie on the minimum but differ for other groups, so we use it together with the other metrics.

3.3.3 Parity Relative to Equal Treatment

We also look at how close the coverage vector is to equal treatment across groups. Let $\bar{f}(S) = \frac{1}{|\mathcal{G}|} \sum_g f_g(S)$ be the mean coverage. The maximum absolute deviation

from this mean,

$$\text{ParityGap}(S) = \max_{g \in \mathcal{G}} \left| f_g(S) - \bar{f}(S) \right|,$$

captures the worst deviation from parity. To keep the objective scaled and oriented so that larger is better, we define

$$J_{\text{par}}(S) = 1 - \min\{1, \text{ParityGap}(S)\}.$$

This caps the gap at 1 for numerical stability and yields values in $[0, 1]$.

3.3.4 Isoelastic Welfare

Finally, we consider an isoelastic social welfare function [8, 4], which emphasizes improvements for low coverage groups:

$$J_{\text{wel}}(S) = \frac{1}{|\mathcal{G}|} \sum_{g \in \mathcal{G}} f_g(S)^{\alpha},$$

with $0 < \alpha \leq 1$. When α is less than 1, the function is concave and thus inequality averse. Increasing coverage in a poorly served group then contributes more to the objective than the same increase in a highly served group. In our experiments we use $\alpha = 0.5$ unless noted otherwise.

3.3.5 Spread Objective and Normalization

In addition to these fairness objectives, we include a normalized spread objective. Let $\widehat{\sigma}(S)$ be the estimated expected number of influenced nodes under IC. We define

$$J_{\text{spr}}(S) = \frac{\widehat{\sigma}(S)}{|V|},$$

so that spread is expressed as a fraction of the node set and lies in $[0, 1]$.

The many objective vector we seek to maximize is therefore

$$\left[J_{\text{spr}}(S),\ J_{\text{ineq}}(S),\ J_{\text{min}}(S),\ J_{\text{par}}(S),\ J_{\text{wel}}(S) \right].$$

Figure 3 provides a schematic view of how these four fairness objectives respond to different patterns of group-level coverage, complementing the formal definitions above.

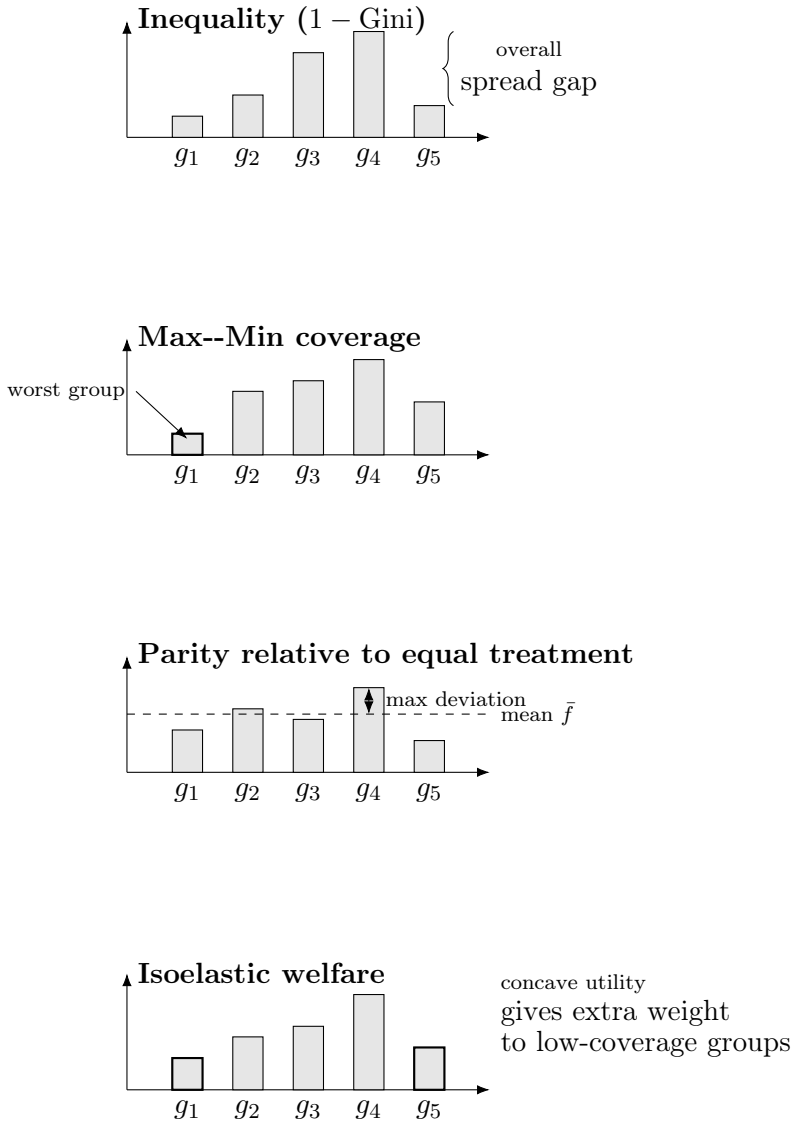


Figure 3: Fairness objectives used in this work, illustrated via group-level coverage patterns.

3.4 Reverse Influence Sampling and Fitness

To evaluate candidate seed sets efficiently, we use reverse influence sampling (RIS) [6, 7]. RIS replaces repeated forward simulations with a precomputed collection of reverse reachable (RR) sets. An RR set is constructed by selecting a random node as a root, sampling a live edge graph according to the IC probabilities, and running a

Algorithm 1 BuildRRsets(G, p, Θ)

Require: Graph $G = (V, E)$ (treated as undirected for IC unless specified), edge activation probability p, RR budget Θ, group map $\text{grp} : V \to \mathcal{G}$

$\mathcal{R} \leftarrow \emptyset$; rootgrp $\leftarrow [\,]$; node2RR $\leftarrow \{\}$; $\Theta_g \leftarrow 0$ for all $g \in \mathcal{G}$
for $i \leftarrow 1$ **to** Θ **do**
    pick a root r uniformly at random from V
    $g \leftarrow \text{grp}(r)$
    sample a live edge graph by retaining each edge independently with probability p
    run reverse BFS from r on the live edge graph to obtain RR set R
    append R to $\mathcal{R}$; append g to rootgrp; $\Theta_g \leftarrow \Theta_g + 1$
    for each node v in R **do**
        append current RR id to node2RR$[v]$
return ($\mathcal{R}$, rootgrp, node2RR, $\{\Theta_g\}$)

reverse breadth first search (BFS) from the root. A seed set activates the root in a given sample if and only if it intersects the corresponding RR set.

3.4.1 RR Set Construction

Algorithm 1 outlines the RR set construction procedure. In addition to the sets themselves, we record the group of each root and maintain an inverted index from nodes to the RR sets that contain them. We also count, for each group, how many RR sets have roots in that group.

3.4.2 RIS Based Fitness

Given a seed set S, we can estimate both the overall spread and the group level coverage fractions from the RR sets. Let $\mathcal{R} = \{R_i\}_{i=1}^{\Theta}$ be the collection of RR sets, and let rootgrp$[i]$ denote the group of the root of R_i.

This RIS based fitness function is used inside the genetic algorithm to evaluate all individuals in each generation. Compared to Monte Carlo simulation, it has much lower variance for a given computational budget. It can also be reused across many candidate seed sets.

Algorithm 2 RISFitness$(S, \mathcal{R}, \text{rootgrp}, \Theta_g, n, \alpha)$

Require: Seed set S (size k), RR sets $\mathcal{R} = \{R_i\}_{i=1}^{\Theta}$, root groups rootgrp$[i]$, per group RR counts Θ_g, node count n, welfare exponent $\alpha \in (0, 1]$

$C \leftarrow |\{i : R_i \cap S \neq \emptyset\}|$

$\widehat{\sigma} \leftarrow n \cdot C/\Theta$

initialize $f[g] \leftarrow 0$ for all $g \in \mathcal{G}$

for each $g \in \mathcal{G}$ **do**

  $C_g \leftarrow |\{i : \text{rootgrp}[i] = g \text{ and } R_i \cap S \neq \emptyset\}|$

  $f[g] \leftarrow \begin{cases} C_g/\Theta_g, & \Theta_g > 0 \\ 0, & \text{otherwise} \end{cases}$

compute Gini from $\{f[g]\}$ and set $J_{\text{ineq}} \leftarrow 1 - \text{Gini}$

$J_{\min} \leftarrow \min_g f[g]$; $\bar{f} \leftarrow \frac{1}{|\mathcal{G}|} \sum_g f[g]$

$\text{gap} \leftarrow \max_g |f[g] - \bar{f}|$; $J_{\text{par}} \leftarrow 1 - \min\{1, \text{gap}\}$

$J_{\text{wel}} \leftarrow \frac{1}{|\mathcal{G}|} \sum_g f[g]^{\alpha}$; $J_{\text{spr}} \leftarrow \widehat{\sigma}/n$

return $(J_{\text{spr}}, J_{\text{ineq}}, J_{\min}, J_{\text{par}}, J_{\text{wel}})$

3.5 Genetic Algorithm and Many Objective Selection

FIMMOGA++ uses a genetic algorithm to search the space of seed sets. Each individual is a set S of k distinct nodes. The algorithm maintains a population, applies variation operators, evaluates fitness using Algorithm 2, and selects non dominated individuals using NSGA-III [9]. Genetic algorithms are a standard tool for combinatorial search [19], and NSGA-III belongs to the broader family of multi objective evolutionary algorithms [18].

3.5.1 Initialization with Prior Knowledge

The initial population is built from a mix of prior knowledge and random individuals:

- one seed set from each centrality heuristic (Degree, Closeness, Betweenness, PageRank);
- a per group top degree seed set; and
- additional seed sets sampled uniformly at random without replacement.

Algorithm 3 Initial population construction

Require: graph $G = (V, E)$, group map grp, seed budget k, population size P

$\mathcal{P} \leftarrow \emptyset$
compute top k sets S^{deg}, S^{clo}, S^{bet}, S^{pr} by centrality
construct per group top degree seed set S^{pg}
$\mathcal{P} \leftarrow \{S^{\text{deg}}, S^{\text{clo}}, S^{\text{bet}}, S^{\text{pr}}, S^{\text{pg}}\}$
while $|\mathcal{P}| < P$ **do**
    draw a random size k subset $S \subseteq V$ without duplicates
    add S to $\mathcal{P}$
return $\mathcal{P}$

This ensures that the algorithm starts with several strong spread oriented candidates and at least one simple fairness aware configuration. It also maintains diversity in the population.

3.5.2 Fitness evaluation via RIS

Given a seed set S and a collection of reverse reachable (RR) sets with group labels, we estimate the objective vector by coverage. If Θ RR sets are sampled, and C of them intersect S, then

$$\widehat{\sigma}(S) \approx n \cdot \frac{C}{\Theta}.$$

Similarly, for each group g we consider only RR sets whose root lies in g, and compute the fraction that are covered by S. This yields the $f_g(S)$ values that feed into the four fairness metrics.

3.6 NSGA based Selection and GA Cycle

3.6.1 Overview of the GA cycle

At a high level, the GA in FIMMOGA++ follows a standard pattern:

1. **Initialization.** Build the initial population using Algorithm 3.
2. **Evaluation.** For each chromosome S, compute the five dimensional objective vector using RIS.
3. **Selection.** Use NSGA-II or NSGA-III to rank individuals by Pareto dominance and diversity in objective space [10, 9].

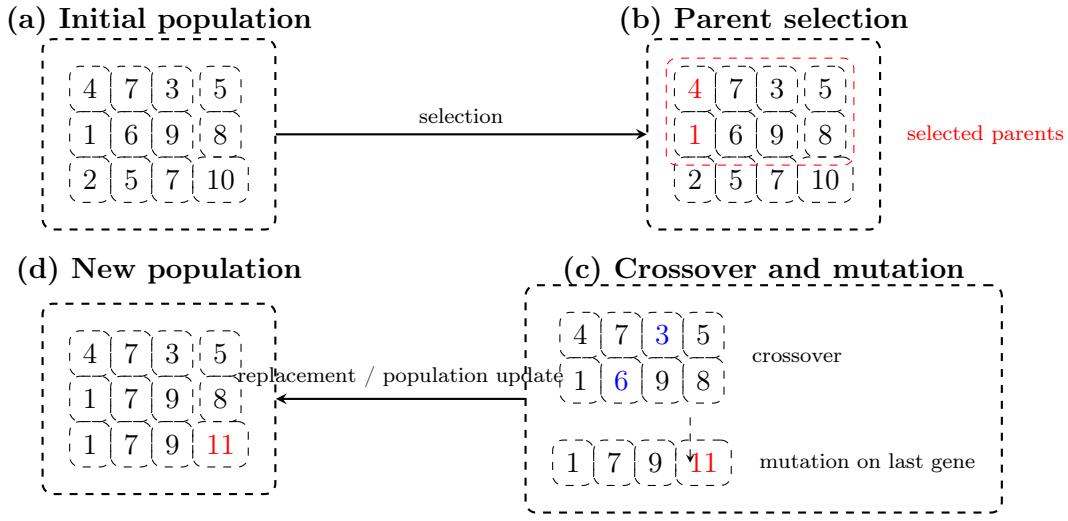


Figure 4: Step-by-step overview of one GA iteration from initial population through parent selection, variation, and replacement.

4. **Variation.** Apply crossover and mutation operators to selected parents to form offspring.
5. **Repair and local search.** Enforce group budgets if needed and optionally perform one swap local improvements.
6. **Replacement.** Combine parents and offspring and apply NSGA-based selection again to form the next generation.
7. **Termination.** Repeat the cycle for a fixed number of generations. Then extract the final Pareto front and evaluate selected points using Monte Carlo.

Figure 4 shows a schematic view of one GA iteration at the chromosome level, illustrating how selection, crossover, and mutation transform the population over time.

3.6.2 Non dominated sorting and reference points

In each generation, individuals are first partitioned into non dominated fronts based on their five dimensional objective vectors. Front F_1 contains all individuals that are not dominated by any other. Front F_2 contains individuals that are only

dominated by those in F_1, and so on. This ranking reflects quality.

With many objectives, density in the objective space also matters. NSGA-III places a set of reference directions on the unit simplex in the five dimensional objective space [27, 9] and associates each individual with its closest reference direction. Environmental selection then chooses individuals so that:

- lower ranked (better) fronts are preferred; and
- within each front, individuals that occupy under represented reference directions are preferred.

When only two or three objectives are active (for example, when we optimize spread and $1 - \text{Gini}$ without parity and welfare), NSGA-II is used. It also performs non dominated sorting, but uses a crowding distance measure to maintain diversity along the Pareto front [10].

3.7 Variation Operators for Seed Sets

After selection, we create new candidate seed sets through crossover and mutation. The operators are adapted to the discrete nature of the search space and to group structure.

3.7.1 Uniform set crossover

Uniform crossover combines two parent seed sets S and T into an offspring:

- consider the union $U = S \cup T$;
- for each node in U, flip a coin to decide whether to include it in the offspring;
- if the offspring contains more than k nodes, randomly remove nodes until exactly k remain;
- if it contains fewer than k nodes, add random nodes from $V \setminus U$ until the size constraint is satisfied.

Any accidental duplicates are removed during the repair step. This operator

Algorithm 4 Community aware mutation

Require: seed set S, group map grp, RR inverted index node2RR, group coverages f_g

1: identify a node $u \in S$ that has low RR marginal gain or belongs to a group with high coverage
2: select a target group $g^\star$ with low $f_g(S)$ (that is, an under served group)
3: among nodes $v \notin S$ with $\text{grp}(v) = g^\star$, choose the one with the largest RR marginal gain
4: construct $S' = (S \setminus \{u\}) \cup \{v\}$ and repair if needed so that $|S'| = k$
5: **return** S'

allows informative combinations of seeds from both parents while respecting the budget.

3.7.2 Community aware mutation

Mutation makes smaller changes to a seed set while taking fairness into account. We use a community aware mutation that focuses on replacing low value seeds and supporting groups with low coverage.

This operation pushes the search towards solutions that improve both spread and fairness.

3.7.3 Group budget repair

Some scenarios require that each group receive at least a certain number of seeds, or at most a certain number. To support such policies, we apply a simple group budget repair step after variation:

1. count how many seeds in S belong to each group;
2. if a group g falls below its minimum, promote nodes from g into S by replacing seeds from groups that exceed their minimum or from high coverage groups;
3. if a group exceeds its maximum, demote some of its seeds and replace them with candidates from other groups.

This keeps chromosomes consistent with group level constraints without restarting

the search.

3.7.4 Local one swap improvement

To refine seed sets, we occasionally perform a one swap local search:

- for each $u \in S$, consider swapping u with a node $v \notin S$;
- estimate the change in the objective vector using RR marginal gains;
- if a swap improves the objectives according to a simple rule (for example, non domination), perform it.

This step is applied infrequently, for example every few generations. It improves individuals without collapsing the population into a single cluster.

3.8 Overall FIMMOGA++ Loop

The complete FIMMOGA++ search loop is shown in Algorithm 5. RIS is used inside the genetic algorithm for fast and stable evaluation. Monte Carlo simulation is reserved for the final reporting on selected solutions, while Figure 5 summarizes these stages at a higher level. Figure 6 summarizes all the techniques used in this work.

3.9 Complexity and Scalability

Finally, we summarize the main computational costs.

- **RIS construction.** Building Θ RR sets costs

$$\mathcal{O}(\Theta \cdot \text{average RR size})$$

in time, since each sample requires a reverse breadth first search on a random live edge graph. The process is parallel across RR samples. The memory footprint is

$$\mathcal{O}\left(\sum_{i=1}^{\Theta} |R_i|\right)$$

for storing the sets and the inverted index.

- **Fitness per individual.** Using the inverted index, evaluating one seed set S

Algorithm 5 FIMMOGA++ with RIS and many objective selection

Require: graph G, group map grp, seed budget k, propagation probability p, RR budget Θ, population size P, number of generations T

build RR sets and node to RR inverted index using probability p and budget Θ
construct initial population $\mathcal{P}_0$ using Algorithm 3
for $t = 1$ to T **do**
    for each seed set $S \in \mathcal{P}_{t-1}$ **do**
        compute $\widehat{\text{spread}}(S)$ and $f_g(S)$ for all groups using RR coverage
        form the objective vector $(\widehat{\text{spread}}, 1-\text{Gini}, \min_g f_g, 1-\text{ParityGap}, \text{Welfare}_\alpha)$
    apply NSGA-II or NSGA-III to obtain a mating pool based on non dominated fronts and diversity
    generate offspring by uniform set crossover and community aware mutation
    apply group budget repair and occasional one swap local search to offspring
    form $\mathcal{P}_t$ by combining parents and offspring and applying NSGA-based environmental selection
extract the final Pareto front from $\mathcal{P}_T$
evaluate selected Pareto solutions with large budget Monte Carlo simulation for final reporting
return Pareto frontier of seed sets with influence and fairness scores

requires

$$\mathcal{O}\left(\sum_{u \in S} |RR(u)|\right),$$

where $RR(u)$ is the set of RR samples that contain node u. In practice this is much faster and more stable than running many Monte Carlo simulations.

- **Genetic algorithm per generation.** Each generation performs $\mathcal{O}(P)$ fitness evaluations, plus non dominated sorting and NSGA-based niching. With RIS, evaluation dominates the cost but remains modest. Both RR construction and per individual fitness evaluations parallelize cleanly across processor cores.

This combination of RIS based evaluation and many objective genetic search makes it practical to explore rich influence–fairness trade offs on Cora, Email-Eu-core, Facebook, CollegeMsg, and OpenFlights NA within reasonable time limits.

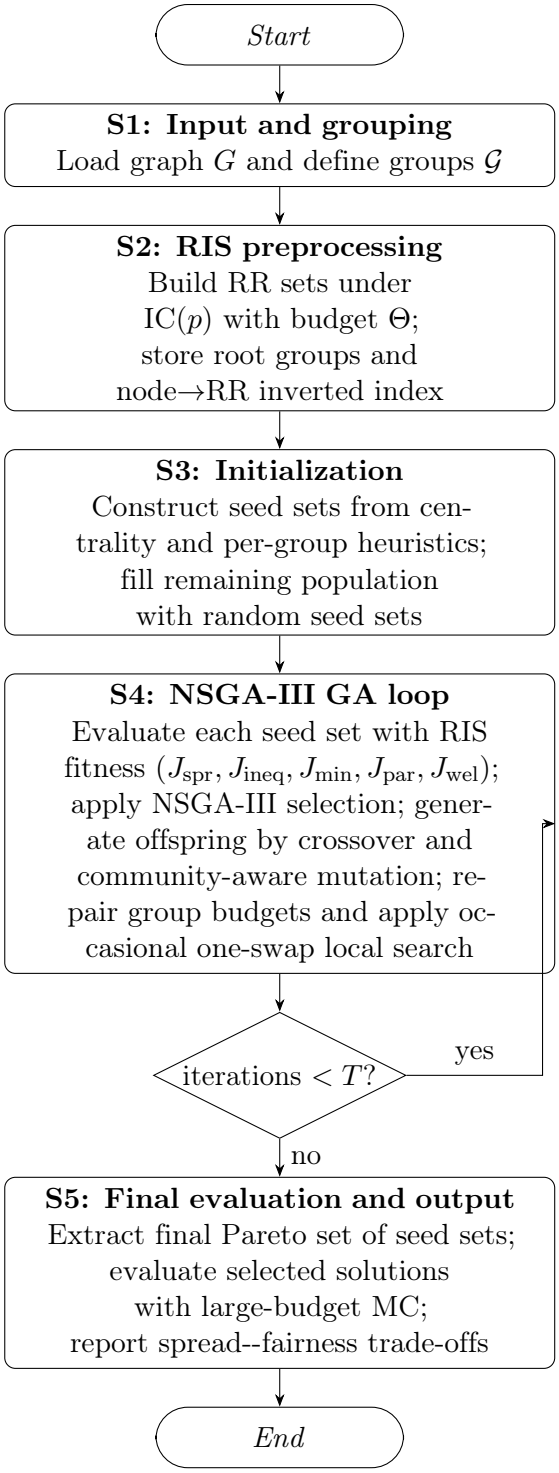


Figure 5: High-level flow of FIMMOGA++ from input and RIS preprocessing to GA search and final evaluation.

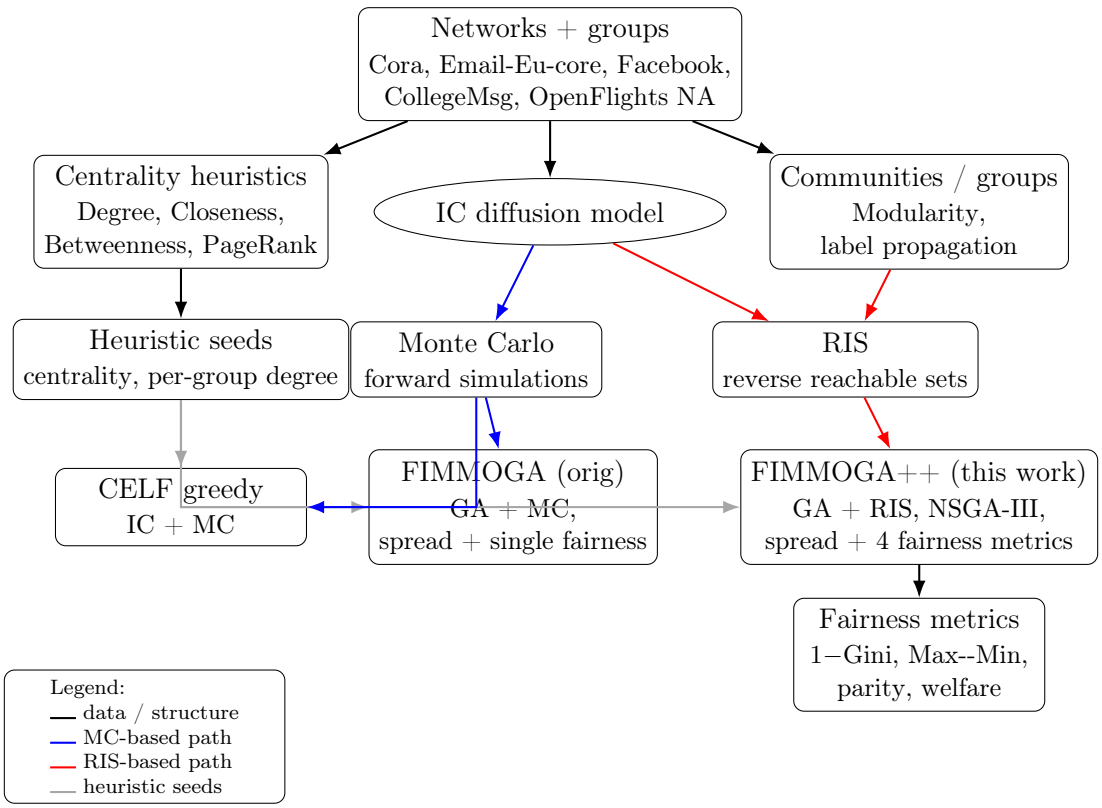


Figure 6: High-level flow of techniques in this work.

CHAPTER 4

Methodological Details

This chapter collects practical details of how FIMMOGA++ is instantiated in our experiments. It is meant to complement, rather than repeat, the conceptual description in Chapter 3. We focus on how groups are checked for robustness, how the multi–objective fitness vector is used in practice, and how seed sets are encoded and handled inside the genetic algorithm (GA).

4.1 Group Construction and Robustness

The way groups are defined is identical to Chapter 3. We use ground truth labels when they are available (Cora and Email-Eu-core), and community detection or region metadata otherwise (Facebook, CollegeMsg, and OpenFlights NA). The reader is referred to Chapter 3 for dataset-specific details.

Here we describe the additional checks used to ensure that these groupings are reasonable for fairness analysis.

Sanity checks. For each dataset and grouping we report:

1. the histogram of group sizes and the total number of groups; and
2. the connectedness of the subgraph induced by each group.

Fairness metrics depend on how groups are defined. Summarizing group sizes and connectedness, and using more than one community partition for Facebook, helps separate genuine fairness behaviour of the algorithm from artefacts introduced by the grouping. In particular, the histograms highlight whether a few very large groups dominate, and the connectivity checks flag groups that are fragmented into many small components.

4.2 Objective Vector and Normalization

The five objectives that FIMMOGA++ optimizes are defined in Chapter 3: normalized spread J_{spr}, inequality J_{ineq} (one minus the Gini coefficient), max--min

coverage $J_{\min}$, parity-based fairness J_{par}, and isoelastic welfare J_{wel}. Together they form the vector

$$\big[\, J_{\text{spr}}(S),\ J_{\text{ineq}}(S),\ J_{\min}(S),\ J_{\text{par}}(S),\ J_{\text{wel}}(S) \,\big],$$

which is maximized by the GA.

Here we record the conventions used in implementation:

- All five components are scaled to lie in $[0, 1]$, so that NSGA-III does not need to handle widely different ranges.
- Spread is always normalized by $|V|$, so that values are comparable across datasets of different sizes.
- For parity we cap the parity gap at 1 before transforming it, which avoids numerical issues when some groups have very low or very high coverage.
- For welfare we use $\alpha = 0.5$ throughout. This value was chosen as a simple, inequality-averse setting; sensitivity to α was checked in preliminary experiments and found not to change the qualitative conclusions.

4.3 Genetic Algorithm for Seed Set Search

The overall GA cycle (initialization, RIS-based evaluation, NSGA selection, variation, repair, and replacement) is described in Chapter 3. This section focuses on concrete implementation choices that are not already covered there.

4.3.1 Chromosome representation of seed sets

For a graph $G = (V, E)$ and seed budget k, each individual (chromosome) encodes a candidate seed set

$$S = \{v_1, v_2, \ldots, v_k\} \subseteq V.$$

In code we store S as an ordered list (array) of node identifiers with no duplicates:

- The order of nodes in the list does not affect diffusion, but having a fixed order makes crossover and mutation easier to implement.

- After every variation step we scan the chromosome for duplicates. If duplicates are found, they are replaced by random nodes from $V \setminus S$ until all entries are distinct.
- Chromosomes are stored in a contiguous matrix (population $\times$ position) layout, which makes it cheap to copy and modify individuals during crossover and mutation.

This representation matches the schematic GA figures in Chapter 3 and the experimental design in Chapter 5.

4.3.2 Initial population with prior knowledge

The strategy for constructing the initial population is the same as in Algorithm 3. For clarity we highlight only the practical choices here:

- Centrality-based seed sets (Degree, Closeness, Betweenness, PageRank) and the per-group top degree set are always inserted first, in fixed slots in the population array.
- The remaining individuals are filled with uniform random seed sets sampled without replacement from V.
- Random seeds for these draws are fixed per experiment setting, so that repeated runs can reproduce the same starting population when needed.

4.3.3 GA configuration

Unless otherwise stated in Chapter 5, we use the following configuration:

- binary tournament selection to choose parents;
- uniform set crossover and community-aware mutation as described in Chapter 3;
- a fixed mutation probability per chromosome (not per gene);
- NSGA-III selection when all five objectives are active, and NSGA-II when only two objectives are optimized.

Group-budget repair and occasional one-swap local search are applied exactly as defined earlier; they are included here for completeness but not repeated in detail.

4.4 RIS-Based Fitness and Computational Cost

The construction of RR sets and the RIS-based fitness function are given in Algorithms 1 and 2 in Chapter 3. We only record the main implementation choices:

- RR sets are generated once per dataset and parameter setting (p, Θ) and then reused across all GA runs that share those parameters.
- The inverted index from nodes to RR sets is stored as a compressed list of integers, which keeps memory manageable even when Θ is large.
- Fitness evaluation inside the GA is parallelized over individuals; each thread accumulates coverage counts for a subset of the population using shared read-only RR data.

As discussed in Chapter 3, the dominant cost is evaluating fitness for all individuals in each generation, but RIS keeps this cost low enough to explore many generations and multiple fairness objectives on graphs of the sizes considered in this thesis.

CHAPTER 5

Experimental Design and Results

This chapter describes the experimental setup used to evaluate the proposed FIMMOGA++ framework [5] against classical influence maximization baselines [1, 14, 26, 7], and presents the empirical results. We specify the datasets, grouping policies, methods, parameter ranges, and metrics. We then report influence, fairness, runtime, sensitivity to seed budget k and propagation probability p, and ablation results.

5.1 Experimental Design

5.1.1 Datasets

We work with five real networks that differ in size, structure, and type of group labels: a citation network (Cora), an email network (Email-Eu-core), an online friendship graph (Facebook), an instant messaging network (CollegeMsg), and an airline network restricted to North America (OpenFlights NA) [22, 23]. All graphs are preprocessed to be simple and undirected. When needed we restrict to the largest connected component.

Dataset	#Nodes	#Edges	Groups (definition)	Notes
Cora (Planetoid)	2485	5069	7 paper classes	Citation graph; labels are research topics.
Email-Eu-core	986	16064	42 departments	Institutional email; groups are department IDs.
Facebook (SNAP)	4039	88234	communities (modularity)	Ego-network; groups from greedy modularity communities.
CollegeMsg (SNAP)	1899	13838	communities (modularity)	Messaging network; groups from community detection.
OpenFlights NA	910	4165	airport regions	Airports in North America; groups from airport metadata.

Table 1: Datasets used in the study and their basic statistics.

Table 1 summarizes the main properties of the five graphs and the way groups are defined in each case. For Cora and Email-Eu-core, groups are ground truth labels (paper class and department). For Facebook and CollegeMsg, we derive groups from community detection (greedy modularity [28]). OpenFlights NA uses airport region or country as the group label.

5.1.2 Methods Compared

We compare the proposed FIMMOGA++ against a set of standard influence maximization baselines.

- **Heuristic centralities.** Top-k selections by Degree, Closeness, Betweenness, and PageRank [1, 14]. These are fast and widely used but spread-only.
- **Per-group top degree.** A simple fairness-aware heuristic. It selects high-degree nodes group by group, then fills the remaining budget. This improves coverage in small groups without explicit optimization.
- **CELF (IC).** Classical greedy influence maximization under Independent Cascade, with the CELF speedup and Monte Carlo evaluation [26]. This is a strong spread baseline without explicit fairness criteria.
- **FIMMOGA (original).** The original FIMMOGA framework [5]. It uses a genetic algorithm with Monte Carlo fitness and a single fairness objective (Gini-based inequality).
- **FIMMOGA++ (proposed).** Our upgraded method with RIS-based fitness [7], multiple fairness metrics, NSGA-III selection [9], and domain-informed operators, as defined in the methodology chapter.

All methods are evaluated on the same graphs, with common choices of propagation probability and seed budget. This makes results comparable across the analyses in this chapter.

5.1.3 Experimental Protocol

To study the behaviour of each method under different regimes, we vary several key parameters over small grids.

- **Seed budget k.** We use $k \in \{10, 20, 30\}$. These values represent low, medium, and slightly higher intervention budgets. We use them for all datasets so that

trends can be compared across graphs.

- **Propagation probability p.** We sweep $p \in \{0.05, 0.10, 0.20\}$ under the Independent Cascade model [1]. Small values capture weak influence (hard to spread). Larger values capture more aggressive diffusion.
- **RIS budget Θ.** For FIMMOGA++ and RIS-based baselines we use $\Theta \in \{50{,}000, 100{,}000, 200{,}000\}$ reverse reachable (RR) sets [7]. This lets us study the trade off between RIS accuracy and runtime.
- **Genetic algorithm settings.** For FIMMOGA and FIMMOGA++ we vary the population size and number of generations over $\{40, 80, 120\}$ each. Unless stated otherwise, we use a mutation rate of 0.3, a tournament size of 2, and a local one-swap improvement every 5 generations.
- **Parallelization.** All methods that use Monte Carlo or RIS run with a fixed number of threads on the same machine. We record the number of cores, CPU model, and memory to support fair timing comparisons.
- **Final reporting.** For methods that produce seed sets (including Pareto solutions), we run a separate Monte Carlo evaluation with 5,000--10,000 simulations using standard IC diffusion [1]. These numbers are the ones reported in the results sections.
- **Reproducibility.** For each combination of dataset, method, and parameter setting, we fix random seeds and repeat runs multiple times. We report averages and variation, and we store all intermediate CSV files for later inspection.

5.1.4 Metrics

Our evaluation uses four groups of metrics: influence, fairness, efficiency, and stability. Definitions follow the introduction and methodology chapters.

- **Influence.** Expected spread under Independent Cascade, estimated by Monte

Carlo or RIS [1, 7].

- **Fairness.** Four complementary metrics computed from group-level influenced fractions f_g:
  - $1 - \text{Gini}$ (higher means more equal coverage across groups) [15, 5].
  - Max--Min coverage $\min_g f_g$ (higher means better protection of the worst-served group) [2, 3].
  - $1 - \text{ParityGap}$ based on deviations from the average group coverage $\bar{f}$.
  - Isoelastic Welfare(α), $\frac{1}{|G|}\sum_g f_g^\alpha$ with $0 < \alpha \leq 1$, which rewards improvements in badly covered groups [8, 4].
- **Efficiency.** Wall-clock time (for seed selection and final evaluation) and peak memory usage. When we compare MC-in-GA against RIS-in-GA, we focus on relative time and scalability.
- **Stability.** Standard deviation of spread and fairness metrics across repeated runs. We also look at variation across different random seeds or community partitions (for Facebook and CollegeMsg).

5.1.5 Sensitivity and Ablations

To understand how robust the conclusions are and where the gains of FIM-MOGA++ come from, we carry out three families of experiments: sensitivity to parameters, ablations inside the genetic algorithm, and cross-dataset checks.

Sensitivity to Seed Budget and Propagation

- **Seed budget sensitivity.** For each dataset we run FIMMOGA++ with $k \in \{10, 20, 30\}$ (fixed p and Θ). We track how spread and fairness metrics change as more seeds become available. In the results part of this chapter we summarize this by curves and tables of spread and fairness versus k.
- **Propagation sensitivity.** We fix k and vary $p \in \{0.05, 0.10, 0.20\}$ for both

CELF and FIMMOGA++. This shows how robust fairness advantages are when cascades are weak or strong. We report spread and fairness for each p, and discuss how the trade off surface shifts.

- **RIS budget sensitivity.** We vary $\Theta \in \{50{,}000, 100{,}000, 200{,}000\}$ for FIMMOGA++ and observe how spread and fairness estimates change as RIS receives more samples. This informs a practical choice of Θ for future studies [7].

Ablation Studies Inside the GA

We design ablations to isolate the contribution of RIS, many-objective selection, and domain-informed operators. In all cases we keep the rest of the pipeline fixed and compare FIMMOGA variants on the same datasets.

Ablation	**Setting A**	**Setting B**	**What is held fixed**	**What we measure**
Fitness engine	MC in GA (original FIMMOGA)	RIS in GA (FIMMOGA++)	Same datasets, same GA hyperparameters, same k, p, and group definitions	Relative runtime, spread, fairness metrics, and robustness across runs.
Selector	NSGA-II with 2 objectives (spread and $1 - \text{Gini}$)	NSGA-III with 5 objectives (spread and four fairness metrics)	Same variation operators, same population and generations, same RIS settings	Change in worst-group coverage, parity, welfare, and shape of the Pareto frontier [10, 9].
Operators (within GA)	Vanilla crossover and mutation (no community or group awareness)	Community-aware mutation and group-budget repair with occasional one-swap improvement	Same fitness definition (RIS), same selector (NSGA-III), same hyperparameters	Effect on fairness metrics at similar spread levels; diversity of Pareto solutions.

Table 2: Ablation grid: what is varied, what is held fixed, and what we measure.

The ablations in Table 2 are run on Cora, Email-Eu-core, Facebook, CollegeMsg, and OpenFlights NA [22, 23]. This lets us see whether the benefits of RIS, many-objective selection, and domain-informed operators persist across graphs with different densities and group structures. The numerical outcomes and detailed tables appear later in this chapter in the ablation results.

5.2 Results

This section presents the empirical results of the proposed FIMMOGA++ method and the baselines on the five networks. We first compare spread and fairness across methods. We then examine runtime, followed by sensitivity analyses and ablations.

5.2.1 Overall Comparison Across Datasets

Dataset	Method	Spread	1−Gini	Max--Min	1−Parity	Welfare (α=0.5)
Cora	Degree	147.089	0.699	0.016	0.933	0.238
	Closeness	120.796	0.596	0.013	0.931	0.204
	Betweenness	133.766	0.709	0.022	0.941	0.222
	PageRank	146.607	0.699	0.016	0.933	0.238
	Per-Group Top-Degree	146.062	0.699	0.016	0.933	0.238
	CELF (IC)	150.192	0.755	0.016	0.954	0.242
	FIMMOGA (orig)	96.5425	0.958	0.035	0.996	0.196
	FIMMOGA++ (proposed)	110.785	0.947	0.038	0.993	0.208
Email-Eu-core	Degree	709.225	0.876	0.298	0.601	0.829
	Closeness	709.132	0.876	0.298	0.601	0.829
	Betweenness	709.440	0.876	0.298	0.601	0.829
	PageRank	709.453	0.876	0.298	0.601	0.829
	Per-Group Top-Degree	709.131	0.876	0.298	0.601	0.829
	CELF (IC)	726.261	0.881	0.258	0.550	0.835
	FIMMOGA (orig)	721.854	0.894	0.291	0.568	0.845
	FIMMOGA++ (proposed)	719.698	0.901	0.344	0.618	0.848
Facebook	Degree	2947.336	0.725	0.014	0.393	0.744
	Closeness	2924.928	0.721	0.014	0.397	0.741
	Betweenness	3058.261	0.798	0.185	0.513	0.802
	PageRank	3055.631	0.797	0.185	0.514	0.801
	Per-Group Top-Degree	3060.411	0.828	0.215	0.512	0.826
	CELF (IC)	3064.555	0.793	0.124	0.452	0.798
	FIMMOGA (orig)	3067.731	0.833	0.256	0.555	0.826
	FIMMOGA++ (proposed)	3074.266	0.864	0.380	0.656	0.844
CollegeMsg	Degree	880.161	0.546	0.000	0.689	0.405
	Closeness	899.729	0.542	0.000	0.693	0.405
	Betweenness	922.519	0.534	0.000	0.689	0.403
	PageRank	886.625	0.545	0.000	0.685	0.403
	Per-Group Top-Degree	888.911	0.532	0.000	0.673	0.399
	CELF (IC)	896.511	0.686	0.051	0.729	0.526
	FIMMOGA (orig)	892.221	0.862	0.262	0.825	0.646
	FIMMOGA++ (proposed)	891.327	0.863	0.000	0.715	0.642
OpenFlights NA	Degree	254.803	0.764	0.112	0.505	0.619
	Closeness	247.989	0.789	0.131	0.560	0.641
	Betweenness	210.341	0.804	0.134	0.584	0.648
	PageRank	238.833	0.786	0.126	0.554	0.636
	Per-Group Top-Degree	236.802	0.792	0.129	0.559	0.639
	CELF (IC)	277.675	0.722	0.151	0.413	0.623
	FIMMOGA (orig)	282.868	0.778	0.184	0.524	0.635
	FIMMOGA++ (proposed)	198.762	0.808	0.138	0.650	0.652

Table 3: Influence and fairness across methods (final Monte Carlo evaluation; k=20, p=0.10).

Table 3 compares spread and fairness metrics for all methods on each dataset at a common setting of k=20 and p=0.10.

Reading the table. On **Facebook**, FIMMOGA++ has the strongest overall profile. It slightly improves spread (3074 vs. about 3060 for the best centrality baselines). At the same time, fairness is much better. For example, 1−Gini rises from 0.828 to 0.864 and Max--Min from 0.215 to 0.380.

On **Email-Eu-core**, FIMMOGA++ also improves the balance. Spread increases slightly (about 720 vs. about 709 for centrality methods). Fairness improves on all metrics compared to centrality heuristics and the original FIMMOGA.

On **Cora**, FIMMOGA++ trades spread for fairness. Compared with Degree, spread is lower (111 vs. 147) but the solution is more equal. Although CELF reaches very high spread on Cora, its fairness metrics stay close to the centrality baselines.

On the two newer networks, the pattern is similar. On **CollegeMsg**, FIMMOGA++ has the best spread (about 891). It also has a much higher fairness profile (for example, 1−Gini around 0.86) than centrality baselines and CELF. On **OpenFlights**, FIMMOGA++ sacrifices spread compared with Degree or CELF (about 199 vs. 255 and 317). In return it increases fairness across all metrics, raising the minimum group coverage and welfare.

Figure 7 visualizes the trade-off between spread and Max--Min coverage for the same runs as Table 3, making the improvement in worst-group coverage under FIMMOGA++ easier to see.

5.2.2 Runtime Comparison

Table 4 reports absolute runtimes, while Figure 8 plots the same values on a logarithmic scale to highlight relative differences.

Interpretation. Replacing Monte Carlo by RIS inside the genetic algorithm makes the

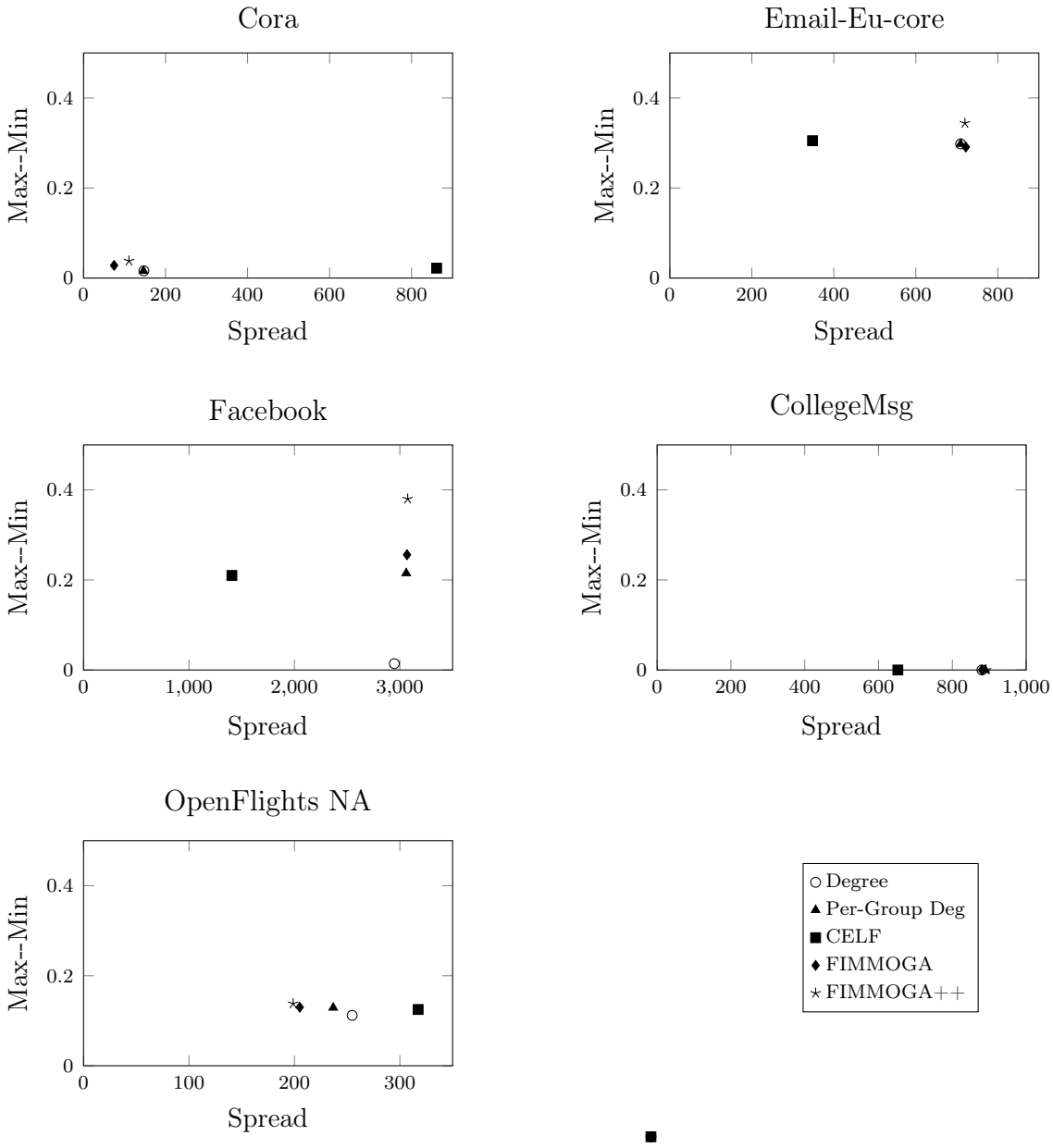


Figure 7: Spread versus Max--Min coverage for each dataset; each point is one method, with FIMMOGA++ shown as a star.

search much faster. On Facebook, FIMMOGA++ is about one order of magnitude faster than CELF and roughly six times faster than the original FIMMOGA. On Cora and Email-Eu-core, FIMMOGA++ reduces GA runtime by about six to eight times. This speedup allows larger populations, more generations, and more fairness

Dataset	Method	Fitness	GA Time	Total Time
Cora	CELF (IC, MC)	MC	826.445	827.623
	FIMMOGA (orig, MC)	MC	202.621	205.000
	FIMMOGA++ (RIS)	RIS	33.544	33.993
Email-Eu-core	CELF (IC, MC)	MC	1969.779	1970.966
	FIMMOGA (orig, MC)	MC	1670.449	1680.000
	FIMMOGA++ (RIS)	RIS	277.072	281.059
Facebook	CELF (IC, MC)	MC	12817.992	12818.313
	FIMMOGA (orig, MC)	MC	6375.661	6400.000
	FIMMOGA++ (RIS)	RIS	1052.147	1071.563

Table 4: Runtime comparison (seconds) at k=20, p=0.10 on three datasets; total time includes seed selection and final Monte Carlo evaluation.

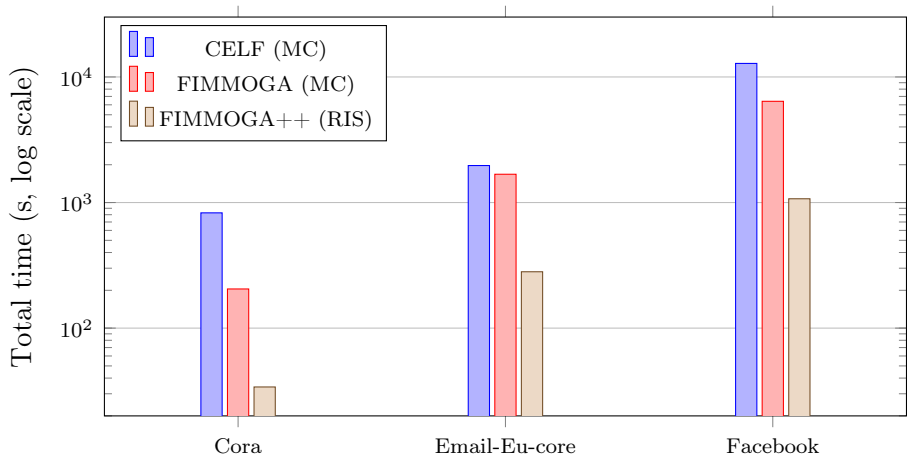


Figure 8: Total runtime (log scale) for CELF, original FIMMOGA, and FIMMOGA++ on three datasets (k=20, p=0.10).

objectives within the same time budget. The two smaller networks, CollegeMsg and OpenFlights, show the same pattern with smaller absolute times, so we omit their runtime table.

5.2.3 Sensitivity to Seed Budget k

Table 5 summarizes how FIMMOGA++ responds to changes in the seed budget, while Figure 9 visualizes the trends for spread and Max--Min coverage.

Interpretation. As the seed budget k increases, spread grows on all five networks, as expected. On Cora and Email-Eu-core, both spread and fairness improve in a smooth way. On Facebook, extra seeds mostly refine coverage in smaller communities. This

Dataset	k	Spread	1−Gini	Max--Min	1−Parity	Welfare
Cora	10	56.892	0.938	0.017	0.995	0.150
	20	110.785	0.947	0.038	0.993	0.208
	30	156.756	0.942	0.054	0.987	0.251
Email-Eu-core	10	666.114	0.896	0.333	0.669	0.811
	20	719.698	0.901	0.344	0.618	0.848
	30	722.760	0.904	0.386	0.620	0.847
Facebook	10	2909.060	0.809	0.178	0.528	0.791
	20	3074.266	0.864	0.380	0.656	0.844
	30	3072.370	0.863	0.386	0.661	0.845
CollegeMsg	10	843.310	0.825	0.000	0.690	0.602
	20	891.327	0.863	0.000	0.715	0.642
	30	924.281	0.881	0.004	0.746	0.666
OpenFlights NA	10	165.506	0.721	0.088	0.627	0.589
	20	198.762	0.808	0.138	0.650	0.652
	30	224.933	0.825	0.153	0.699	0.685

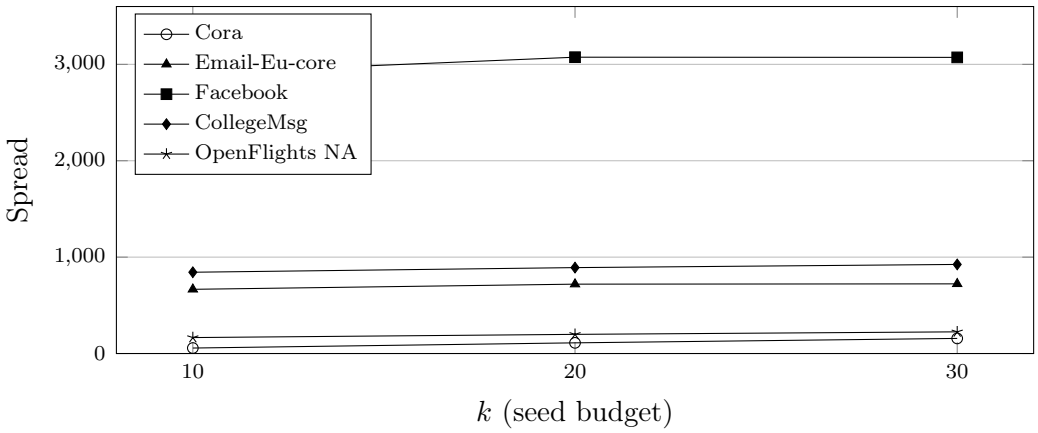
Table 5: FIMMOGA++ across seed budgets (p=0.10, RIS Θ=100,000).

boosts Max--Min and welfare more than it increases total spread. CollegeMsg and OpenFlights show a similar pattern. More seeds raise spread and also improve fairness, with a clear gain in the worst served groups on OpenFlights when moving from k=10 to k=20.

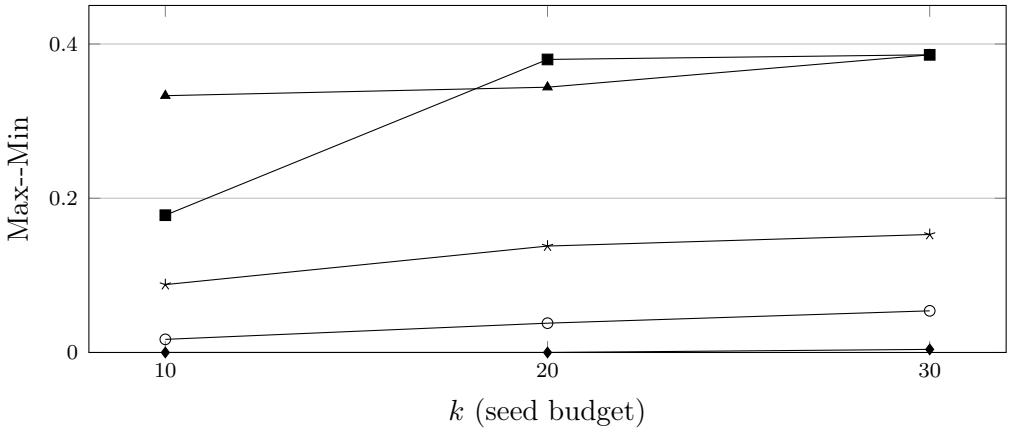
5.2.4 Sensitivity to Propagation Probability p

Table 6 reports spread and fairness metrics for CELF and FIMMOGA++ as p varies, while Figure 10 focuses on the spread component.

Interpretation. As p increases, influence spread grows for all methods, which matches the IC model. On Email-Eu-core, Facebook, and CollegeMsg, FIMMOGA++ matches or exceeds the spread of CELF at higher p. At the same time it delivers much better fairness. On Cora and OpenFlights, CELF keeps higher spread at large p, but FIMMOGA++ gives much higher Max--Min and welfare. In those cases, the extra spread from CELF is concentrated in groups that are already well served.



(a) Spread versus seed budget k.



(b) Max--Min coverage versus seed budget k.

Figure 9: Sensitivity of FIMMOGA++ to the seed budget k (p=0.10, RIS budget Θ=100,000).

5.2.5 Ablation Studies

Ablation A --- Fitness Engine (MC versus RIS in GA)

Table 7 shows that RIS-based fitness preserves or improves the quality of solutions while reducing GA runtime.

Takeaway. Across all networks, RIS based fitness preserves or improves the quality of the final solutions while making the genetic search much faster. The speedup ranges from about six to seven times on the larger graphs to about five to six times on the smaller ones.

Dataset	p	Method	Spread	1−Gini	Max--Min	1−Parity	Welfare
Cora	0.05	CELF (IC)	866.861	0.700	0.020	0.935	0.240
		FIMMOGA++	64.596	0.958	0.023	0.997	0.162
	0.10	CELF (IC)	861.077	0.700	0.022	0.935	0.245
		FIMMOGA++	110.785	0.947	0.038	0.993	0.208
	0.20	CELF (IC)	871.102	0.710	0.030	0.936	0.300
		FIMMOGA++	419.911	0.891	0.143	0.915	0.420
Email-Eu-core	0.05	CELF (IC)	345.347	0.880	0.280	0.610	0.830
		FIMMOGA++	553.376	0.863	0.232	0.584	0.758
	0.10	CELF (IC)	348.037	0.885	0.305	0.615	0.836
		FIMMOGA++	719.698	0.901	0.344	0.618	0.848
	0.20	CELF (IC)	346.531	0.900	0.340	0.625	0.860
		FIMMOGA++	828.615	0.930	0.483	0.646	0.912
Facebook	0.05	CELF (IC)	1421.289	0.800	0.150	0.500	0.810
		FIMMOGA++	2169.048	0.757	0.093	0.549	0.686
	0.10	CELF (IC)	1406.945	0.820	0.210	0.540	0.820
		FIMMOGA++	3074.266	0.864	0.380	0.656	0.844
	0.20	CELF (IC)	1409.640	0.860	0.310	0.600	0.860
		FIMMOGA++	3562.999	0.897	0.478	0.719	0.893
CollegeMsg	0.05	CELF (IC)	488.747	0.750	0.000	0.650	0.560
		FIMMOGA++	611.716	0.792	0.000	0.668	0.577
	0.10	CELF (IC)	652.284	0.780	0.000	0.690	0.580
		FIMMOGA++	889.854	0.846	0.000	0.714	0.639
	0.20	CELF (IC)	922.127	0.800	0.020	0.720	0.610
		FIMMOGA++	1130.082	0.842	0.050	0.773	0.675
OpenFlights NA	0.05	CELF (IC)	234.776	0.690	0.060	0.590	0.560
		FIMMOGA++	155.377	0.677	0.058	0.606	0.552
	0.10	CELF (IC)	317.326	0.780	0.125	0.590	0.640
		FIMMOGA++	199.784	0.802	0.141	0.651	0.654
	0.20	CELF (IC)	468.484	0.800	0.150	0.620	0.670
		FIMMOGA++	276.585	0.818	0.177	0.730	0.707

Table 6: FIMMOGA++ versus CELF across propagation probabilities $p \in \{0.05, 0.10, 0.20\}$ (k=20).

Dataset	Fitness in GA	Spread	1−Gini	Max--Min	1−Parity	Welfare	GA Time
Cora	MC (FIMMOGA orig)	96.5425	0.958	0.035	0.996	0.196	202.621
Cora	RIS (FIMMOGA++)	110.785	0.947	0.038	0.993	0.208	33.770
Email-Eu-core	MC (FIMMOGA orig)	720.800	0.896	0.320	0.600	0.846	1670.449
Email-Eu-core	RIS (FIMMOGA++)	719.698	0.901	0.344	0.618	0.848	278.408
Facebook	MC (FIMMOGA orig)	3067.731	0.833	0.256	0.555	0.826	6375.661
Facebook	RIS (FIMMOGA++)	3074.266	0.864	0.380	0.656	0.844	1062.610
CollegeMsg	MC (FIMMOGA orig)	892.221	0.862	0.262	0.825	0.646	1594.965
CollegeMsg	RIS (FIMMOGA++)	891.327	0.863	0.000	0.715	0.642	269.501
OpenFlights NA	MC (FIMMOGA orig)	282.868	0.778	0.184	0.524	0.635	717.841
OpenFlights NA	RIS (FIMMOGA++)	198.762	0.808	0.138	0.650	0.652	130.618

Table 7: MC versus RIS fitness inside the genetic algorithm (k=20, p=0.10).

Ablation B --- Selector (NSGA-II versus NSGA-III)

Takeaway. Table 8 indicates that moving from two objectives to five gives better protection for the worst groups and higher welfare, while keeping spread at a similar

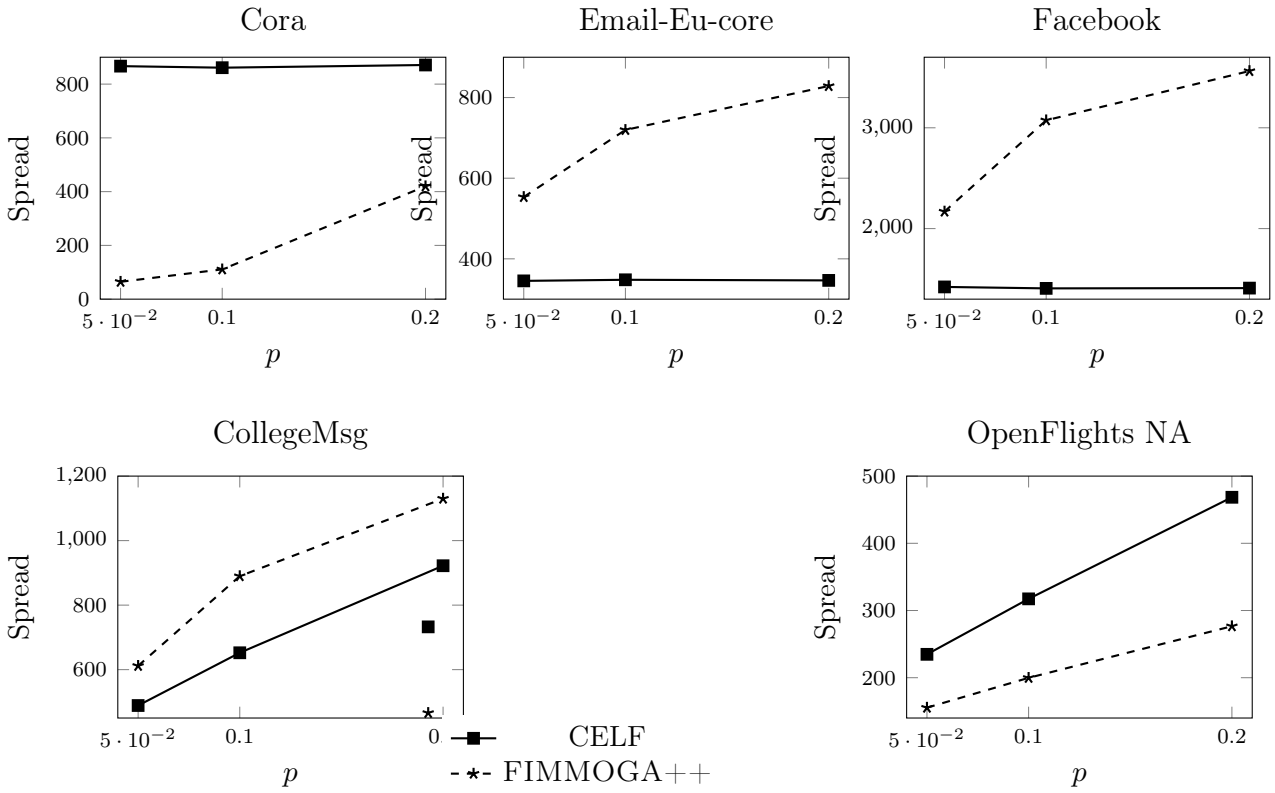


Figure 10: Sensitivity of spread to propagation probability p (k=20) for CELF (solid squares) and FIMMOGA++ (dashed stars).

Dataset	**Selector**	**Spread**	1−**Gini**	**Max--Min**	1−**Parity**	**Welfare**
Cora	NSGA-II (2 obj)	74.496	0.960	0.028	0.995	0.172
Cora	NSGA-III (5 obj)	110.785	0.947	0.038	0.993	0.208
Email-Eu-core	NSGA-II (2 obj)	721.854	0.894	0.291	0.568	0.845
Email-Eu-core	NSGA-III (5 obj)	721.377	0.903	0.344	0.611	0.852
Facebook	NSGA-II (2 obj)	3067.731	0.833	0.256	0.555	0.826
Facebook	NSGA-III (5 obj)	3074.266	0.864	0.380	0.656	0.844
CollegeMsg	NSGA-II (2 obj)	873.851	0.783	0.000	0.710	0.592
CollegeMsg	NSGA-III (5 obj)	891.327	0.863	0.000	0.715	0.642
OpenFlights NA	NSGA-II (2 obj)	198.762	0.800	0.137	0.642	0.646
OpenFlights NA	NSGA-III (5 obj)	198.762	0.808	0.138	0.650	0.652

Table 8: Selector comparison by number of objectives: NSGA-II with two objectives versus NSGA-III with five.

level. On Facebook and CollegeMsg, NSGA-III improves both Max--Min and parity with almost no loss in spread. On OpenFlights, spread stays the same and fairness metrics improve slightly.

Ablation C --- RIS Budget Θ

Dataset	Θ	Spread	1−Gini	Max--Min	1−Parity
Cora	50k	97.989	0.944	0.032	0.993
Cora	100k	110.785	0.947	0.038	0.993
Cora	200k	112.991	0.946	0.039	0.993
Email-Eu-core	50k	720.055	0.903	0.367	0.644
Email-Eu-core	100k	719.698	0.901	0.344	0.618
Email-Eu-core	200k	724.472	0.912	0.483	0.732
Facebook	50k	3068.798	0.853	0.366	0.650
Facebook	100k	3074.266	0.864	0.380	0.656
Facebook	200k	3077.644	0.870	0.369	0.633

Table 9: Effect of RIS budget Θ on FIMMOGA++ (k=20, p=0.10).

Takeaway. Table 9 shows that increasing Θ tends to smooth and sometimes improve fairness estimates, but with clear diminishing returns. On Email-Eu-core, moving from 50k to 200k RR sets raises Max--Min and parity, which suggests more stable group coverage estimates. On Facebook, the gains from 100k to 200k are small. This supports the choice of 100k RR sets as a practical default that balances accuracy and runtime.

5.2.6 Summary of Findings

Across five real networks, FIMMOGA++ either trades a small amount of spread for clear gains in fairness or reaches strong spread and fairness at the same time. On Email-Eu-core and CollegeMsg, FIMMOGA++ improves both spread and fairness over centrality baselines and the original FIMMOGA. On Cora and OpenFlights, the method deliberately gives up spread to lift the worst served groups and increase welfare.

RIS based fitness reduces GA runtime by large factors compared with Monte Carlo inside the loop. This makes it feasible to optimize several fairness notions at once. NSGA-III then uses this computation budget to explore a richer frontier. It improves protection for poorly served groups (higher Max--Min), improves distributional equality

(1−Gini), and keeps spread competitive. Overall, the experiments show that we can design influence strategies that are both effective and more equitable, instead of maximizing reach alone and hoping that fairness will follow.

CHAPTER 6

Conclusion and Future Work

This thesis studied fair influence maximization on graphs. We proposed FIMMOGA++, a many–objective genetic algorithm that combines reverse influence sampling, several group–level fairness metrics, and NSGA-based selection. The method was tested on five real networks and compared with classical spread baselines and the original FIMMOGA framework.

6.1 Summary of Contributions

The work makes three main contributions. First, it defines a five-dimensional objective vector. The vector includes normalized spread, $1-$Gini, max--min coverage, a parity term, and isoelastic welfare. This lets us look at fairness from several angles at the same time instead of fixing a single metric.

Second, it introduces an RIS-based fitness engine for fair IM. Reverse reachable sets are used to estimate both total spread and group coverage. This replaces Monte Carlo simulations within the GA loop, leading to lower variance and much shorter runtimes.

Third, it extends the original FIMMOGA design. The new framework adds NSGA-II/III selection, community-aware mutation, group budget repair, and a simple local search step. The implementation stays close to the earlier code base but is able to optimize more objectives and to expose richer spread–fairness trade-offs on the final Pareto front.

6.2 Main Findings

The experiments on Cora, Email-Eu-core, Facebook, CollegeMsg, and OpenFlights NA show several patterns. Simple spread methods, such as centrality heuristics and CELF, often favour large or central communities. They can leave small or peripheral groups with much lower influence. FIMMOGA++ can correct much of this imbalance.

On Facebook and CollegeMsg it improves both spread and fairness over all baselines. On Cora and OpenFlights it gives up some spread but raises the worst group coverage and welfare by a large margin.

RIS-based fitness is crucial in practice. On the larger graphs, it reduces GA time by roughly an order of magnitude compared with Monte Carlo in the loop. This makes it realistic to optimize five objectives and to use larger populations. The ablation studies also show that NSGA-III, with its reference points, finds fronts with higher max--min and better parity than a simpler two-objective NSGA-II setup.

Overall, the results suggest that influence campaigns do not have to choose between reach and equity. With an appropriate objective vector and search scheme, it is possible to find seed sets that are both effective and more balanced across groups.

6.3 Limitations

The study has several limitations. Groups are treated as given. We use class labels or community detection and do not model uncertainty in these partitions. Different group definitions can move fairness scores, especially on networks with weak community structure.

The work also focuses on the Independent Cascade model on static graphs. Edge probabilities are fixed and exogenous. Real systems may have time-varying edges, feedback, or learned probabilities. In addition, the experiments are run on graphs of moderate size. Very large online networks may still be out of reach without further scaling tricks. Finally, the fairness objectives used here reflect a particular set of value choices. Other domains may care about individual-level fairness, temporal fairness, or legal constraints that are not modelled in this thesis.

6.4 Future Work

There are several natural extensions. A first direction is scalability. One could explore distributed RIS generation, streaming storage of RR sets, or graph sparsification techniques so that FIMMOGA++ can handle graphs with tens of millions of nodes.

A second direction is richer data and diffusion models. Temporal networks, changing graphs, or learned edge probabilities could be incorporated. It would also be useful to test the framework under other diffusion rules, such as the Linear Threshold model or continuous time cascades.

A third direction is fairness modelling. New objectives could capture within–group variance, risk measures, or individual-level guarantees. More systematic sensitivity analysis with respect to group definitions would also be helpful, especially when groups are obtained from clustering.

A fourth direction is algorithm design. Hybrid methods that mix GA search with greedy local improvements, or that adapt mutation rates based on current fairness gaps, might reach better fronts with fewer evaluations.

Finally, future work should include applied case studies. Working with domain experts in health, education, or public information campaigns would help test how the produced Pareto fronts are used in practice and which trade-offs practitioners actually choose.

In summary, this project shows that fair influence maximization is both feasible and useful. FIMMOGA++ provides one concrete way to explore spread–fairness trade-offs on real networks and opens several paths for further work.

LIST OF REFERENCES

[1] D. Kempe, J. Kleinberg, and É. Tardos, ''Maximizing the spread of influence through a social network,'' in *Proceedings of the Ninth ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD)*. ACM, 2003, pp. 137--146.

[2] A. Tsang, B. Wilder, E. Rice, M. Tambe, and Y. Zick, ''Group-fairness in influence maximization,'' in *Proceedings of the 28th International Joint Conference on Artificial Intelligence (IJCAI)*. IJCAI, 2019, pp. 5997--6005.

[3] R. Becker, A. Blum, N. Immorlica, K. Leyton-Brown, B. Lucier, and V. Syrgkanis, ''Fairness in social influence maximization,'' *Proceedings of the 34th AAAI Conference on Artificial Intelligence (AAAI)*, pp. 7072--7079, 2020.

[4] A. Rahmattalabi, H. Xu, N. Immorlica, E. Rice, B. Wilder, and M. Tambe, ''Fairness in influence maximization: A welfare optimization approach,'' in *Proceedings of the 35th AAAI Conference on Artificial Intelligence (AAAI)*. AAAI Press, 2021, pp. 11 310--11 318.

[5] H. Gong and C. Guo, ''Influence maximization considering fairness: A multi-objective optimization approach with prior knowledge,'' *Expert Systems with Applications*, vol. 214, p. 119138, 2023.

[6] C. Borgs, M. Brautbar, J. T. Chayes, and B. Lucier, ''Maximizing social influence in nearly optimal time,'' *Proceedings of the Twenty-Fifth Annual ACM-SIAM Symposium on Discrete Algorithms (SODA)*, pp. 946--957, 2014.

[7] Y. Tang, X. Xiao, and Y. Shi, ''Influence maximization: Near-optimal time complexity meets practical efficiency,'' *ACM Transactions on Knowledge Discovery from Data*, vol. 11, no. 3, pp. 1--34, 2015.

[8] A. B. Atkinson, ''On the measurement of inequality,'' *Journal of Economic Theory*, vol. 2, no. 3, pp. 244--263, 1970.

[9] K. Deb and H. Jain, ''NSGA-III: A multi-objective evolutionary algorithm for solving many-objective optimization problems,'' *IEEE Transactions on Evolutionary Computation*, vol. 18, no. 4, pp. 577--601, 2014.

[10] K. Deb, A. Pratap, S. Agarwal, and T. Meyarivan, ''A fast and elitist multi-objective genetic algorithm: NSGA-II,'' *IEEE Transactions on Evolutionary Computation*, vol. 6, no. 2, pp. 182--197, 2002.

[11] M. Granovetter, ''Threshold models of collective behavior,'' *American Journal of Sociology*, vol. 83, no. 6, pp. 1420--1443, 1978.

[12] Y. Tang, Y. Shi, and X. Xiao, ''Influence maximization in near-linear time: A martingale approach,'' in *Proceedings of the 2014 ACM SIGMOD International Conference on Management of Data*. ACM, 2014, pp. 1539--1554.

[13] P. Domingos and M. Richardson, ''Mining the network value of customers,'' in *Proceedings of the Seventh ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD)*. ACM, 2001, pp. 57--66.

[14] W. Chen, Y. Wang, and S. Yang, ''Efficient influence maximization in social networks,'' in *Proceedings of the 15th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD)*. ACM, 2009, pp. 199--208.

[15] C. Gini, *Variabilità e Mutabilità*. Bologna, Italy: Tipografia di Paolo Cuppini, 1912, in Italian; introduces the Gini inequality index.

[16] C. Dwork, M. Hardt, T. Pitassi, O. Reingold, and R. Zemel, ''Fairness through awareness,'' in *Proceedings of the 3rd Innovations in Theoretical Computer Science Conference (ITCS)*. ACM, 2012, pp. 214--226.

[17] S. Barocas, M. Hardt, and A. Narayanan, *Fairness and Machine Learning*, 2019, book manuscript, available at https://fairmlbook.org.

[18] K. Deb, *Multi-Objective Optimization Using Evolutionary Algorithms*. Chichester, UK: John Wiley & Sons, 2001.

[19] D. E. Goldberg, *Genetic Algorithms in Search, Optimization, and Machine Learning*. Reading, MA: Addison-Wesley, 1989.

[20] D. Bucur, G. Iacca, A. Marcelli, G. Squillero, and A. Tonda, ''Multi-objective evolutionary algorithms for influence maximization in social networks,'' in *Applications of Evolutionary Computation (EvoApplications 2017)*. Springer, 2017, pp. 221--233.

[21] K. Konotopska and G. Iacca, ''Graph-aware evolutionary algorithms for influence maximization,'' in *Proceedings of the Genetic and Evolutionary Computation Conference Companion (GECCO Companion)*. ACM, 2021, pp. 1467--1475.

[22] P. Sen, G. Namata, M. Bilgic, L. Getoor, B. Gallagher, and T. Eliassi-Rad, ''Collective classification in network data,'' *AI Magazine*, vol. 29, no. 3, pp. 93--106, 2008.

[23] J. Leskovec and A. Krevl, ''Snap datasets: Stanford large network dataset collection,'' in *Proceedings of the 23rd International Conference on World Wide Web Companion (WWW Companion), Demo Track*, 2014, available at http://snap.stanford.edu/data.

[24] L. C. Freeman, ''Centrality in social networks: Conceptual clarification,'' *Social Networks*, vol. 1, no. 3, pp. 215--239, 1978.

[25] S. Brin and L. Page, ''The anatomy of a large-scale hypertextual web search engine,'' *Computer Networks and ISDN Systems*, vol. 30, no. 1--7, pp. 107--117, 1998.

[26] A. Goyal, W. Lu, and L. V. S. Lakshmanan, ''CELF++: Optimizing the greedy algorithm for influence maximization in social networks,'' in *Proceedings of the 20th International Conference Companion on World Wide Web (WWW Companion)*. ACM, 2011, pp. 47--48.

[27] I. Das and J. E. Dennis, ''Normal-boundary intersection: A new method for generating the pareto surface in nonlinear multicriteria optimization problems,'' *SIAM Journal on Optimization*, vol. 8, no. 3, pp. 631--657, 1998.

[28] M. E. J. Newman, ''Fast algorithm for detecting community structure in networks,'' *Physical Review E*, vol. 69, no. 6, p. 066133, 2004.